RAF HRIS Backend Implementation Contract

Purpose

This document is the backend-focused split of the master HRIS contract.

This file is intended to stand on its own for backend delivery.

Use this file for: - Laravel application structure and delivery rules - endpoint implementation ownership - request and resource naming standards - migration and seeder planning - RBAC and permission seeding

Master reference for full cross-team context and full payload examples: - [HRIS_API_Architecture_And_Contracts.md]

Table of Contents

- Technology Alignment
- Shared Cross-Client Rules
- Backend Architecture Rules
- Standard API Envelope
- Shared Resource Shapes
- Core Backend ERD
- Endpoint Module Ownership
- Laravel Request and Resource Class Matrix
- Database Migration and Seeder Matrix
- Permission Seed Matrix
- Delivery Notes

Technology Alignment

Backend

- Laravel 12 style application structure
- PostgreSQL
- Laravel Sanctum for API token authentication
- Spatie Laravel Permission for RBAC
- iamfarhad/laravel-audit-log for audit logging
- Form Requests for validation
- Policies for authorization
- API Resources for response serialization
- Actions or Services for business logic

Shared Cross-Client Rules

Authentication Strategy

- All clients authenticate against Laravel API using Sanctum token-based authentication.
- Backend returns a bearer token after successful login.
- Frontend stores the token in a secure app session layer and never duplicates auth logic in every page.
- Mobile stores the token in secure storage and injects it through a shared Dio interceptor.

Authorization Strategy

- Authorization is enforced only by backend policies and scope filters.
- Frontend and mobile may hide unavailable actions for UX, but backend remains the source of truth.
- Permissions are returned in the auth profile payload and used only for presentation logic.

Time and Date Strategy

- All timestamps are ISO 8601 in UTC.
- Date-only values use `YYYY-MM-DD`.
- Clients format dates locally for display only.

Enum Strategy

- All enum-like fields use snake_case strings.
- Example: `pending`, `approved`, `rejected`, `active`, `inactive`.

Pagination Strategy

- All list endpoints return the same pagination envelope.
- Default `per_page` is `20`.
- Maximum `per_page` is `100`.

Error Strategy

- Validation errors always return HTTP `422`.
- Forbidden actions always return HTTP `403`.
- Missing resources always return HTTP `404`.
- Unexpected failures always return HTTP `500` with no internal trace data exposed.

Backend Architecture Rules

Controller Rule

- Controllers only map request to action and return API Resource responses.
- Controllers must not contain business rules, direct query building, or status transition logic.

Validation Rule

- Each write endpoint uses a dedicated Form Request.
- Shared validators are allowed only for truly identical payloads.

Authorization Rule

- Use Policies for entity-level authorization.
- Use query scope filters for list visibility by branch and department.
- Do not hard-code access rules in controllers.

Domain Rule

- Business actions such as approve, reject, check-in, check-out, move stage, run payroll, and run report must be handled by dedicated action or service classes.

Transaction Rule

- Multi-table mutations must use database transactions.
- Approval actions, balance adjustments, and payroll publication must always be transactional.

Response Rule

- Always serialize responses through API Resources or DTO transformers.
- Never return raw Eloquent models directly.

Audit Rule

- Audit capture must occur in shared infrastructure or model observers where appropriate.
- Controllers must not manually assemble audit payloads inline.

Standard API Envelope

Success Response

```
{
  "message": "Request completed successfully.",
  "data": {},
  "meta": {}
}
```

Paginated Response

```
{
  "message": "Request completed successfully.",
  "data": [],
  "meta": {
    "page": 1,
    "per_page": 20,
    "total": 120,
    "last_page": 6
  }
}
```

Validation Error Response

```
{
  "message": "The given data was invalid.",
  "errors": {
    "email": ["The email field is required."],
    "password": ["The password field is required."]
  }
}
```

Forbidden Response

```
{
  "message": "You are not allowed to perform this action."
}
```

Not Found Response

```
{
  "message": "Resource not found."
}
```

Shared Resource Shapes

Employee Summary

```
{
  "id": 101,
  "employee_code": "EMP-000101",
  "full_name": "John Doe",
  "email": "john.doe@raf.local",
  "status": "active",
  "branch": {
    "id": 1,
    "name": "HQ"
  },
  "department": {
    "id": 4,
    "name": "HR"
  },
  "job_title": {
    "id": 7,
    "name": "HR Manager"
  }
}
```

Branch Summary

```
{
  "id": 1,
```

```
  "code": "HQ",
  "name": "Headquarters",
  "status": "active"
}
```

Department Summary

```
{
  "id": 4,
  "name": "Human Resources",
  "parent_id": null,
  "branch_id": 1,
  "status": "active"
}
```

Notification Summary

```
{
  "id": 501,
  "type": "leave_request_approved",
  "title": "Leave approved",
  "body": "Your leave request has been approved.",
  "is_read": false,
  "created_at": "2026-03-11T08:00:00Z"
}
```

Core Backend ERD

Modeling Notes

- Use Spatie tables for role and permission assignment.
- Keep employee home assignment separate from branch and department access scope.
- Prefer lean transactional tables with indexed foreign keys over duplicated summary tables.
- Keep configuration-like rules in dedicated definition tables and only use JSON where flexibility is required.

Core Entity Groups

- Organization and HR core: `branches`, `departments`, `job_titles`, `employees`, `contracts`
- Access control and scope: `roles`, `permissions`, `role_has_permissions`, `model_has_roles`, `model_has_permissions`, `employee_branches`, `employee_departments`
- Operations: `attendance_records`, `shift_templates`, `shifts`, `shift_swap_requests`, `tasks`, `leave_types`, `leave_balances`, `leave_requests`, `overtime_requests`
- Recruitment and lifecycle: `job_postings`, `applicants`, `job_applications`, `application_stage_history`, `onboarding_checklists`, `onboarding_tasks`, `offboarding_checklists`, `offboarding_tasks`
- Planning, talent, learning, payroll, and reporting: `headcount_plans`, `position_forecasts`, `competencies`, `competency_assessments`, `kpi_definitions`, `scorecards`, `kpi_scores`, `training_courses`, `training_enrollments`, `payroll_runs`, `payroll_run_items`, `notifications`, `report_definitions`, `report_runs`

Relational Data Dictionary

Organization and Access Control

Table	Core Columns	Indexes	Constraints	Relations
`branches`	`id`, `code`, `name`, `status`, timestamps	unique `code`, index `status`	`code` required and unique	parent of `departments`, home branch for `employees`, planning and payroll branch anchor
`departments`	`id`, `branch_id`, `parent_id`, `name`, `status`, timestamps	index `branch_id`, `parent_id`, `status`	FK `branch_id -> branches.id`, nullable FK `parent_id -> departments.id`	child of `branches`, parent of `job_titles`, home department for `employees`
`job_titles`	`id`, `department_id`, `name`, `status`, timestamps	index `department_id`, `status`	FK `department_id -> departments.id`	child of `departments`, referenced by `employees`, `headcount_plans`, `position_forecasts`
`employees`	`id`, `employee_code`, `full_name`, `email`, `phone`, `branch_id`, `department_id`, `job_title_id`, `hire_date`, `status`, timestamps	unique `employee_code`, unique `email`, index `branch_id`, `department_id`, `job_title_id`, `status`	FKs to `branches`, `departments`, `job_titles`; `status` constrained to approved enum set	parent of most transactional HR tables

`contracts`	`id`, `employee_id`, `type`, `start_date`, `end_date`, `salary`, `currency`, `status`, timestamps	index `employee_id`, `status`, `start_date`	FK `employee_id -> employees.id`; one active contract per employee enforced by service rule or partial index	child of `employees`
`roles`	Spatie standard columns: `id`, `name`, `guard_name`	unique `name + guard_name`	Spatie-managed uniqueness	many-to-many with permissions and models
`permissions`	Spatie standard columns: `id`, `name`, `guard_name`	unique `name + guard_name`	Spatie-managed uniqueness	many-to-many with roles and models
`role_has_permissions`	`permission_id`, `role_id`	composite PK and lookup indexes per Spatie	FK `permission_id -> permissions.id`, FK `role_id -> roles.id`	joins `roles` to `permissions`
`model_has_roles`	`role_id`, `model_type`, `model_id`	composite PK/index per Spatie	FK `role_id -> roles.id`	maps employees to roles
`model_has_permissions`	`permission_id`, `model_type`, `model_id`	composite PK/index per Spatie	FK `permission_id -> permissions.id`	direct permission grants to employees
`employee_branches`	`id`, `employee_id`, `branch_id`, timestamps	unique `employee_id + branch_id`, index `branch_id`	FK `employee_id -> employees.id`, FK `branch_id -> branches.id`	branch access scope extension for employees
`employee_departments`	`id`, `employee_id`, `department_id`, timestamps	unique `employee_id + department_id`, index `department_id`	FK `employee_id -> employees.id`, FK `department_id -> departments.id`	department access scope extension for employees

Operations and Workforce Transactions

Table	Core Columns	Indexes	Constraints	Relations
`shift_templates`	`id`, `name`, `start_time`, `end_time`, `break_minutes`, `status`, timestamps	index `status`	`break_minutes >= 0`; status enum	parent of `shifts`
`shifts`	`id`, `employee_id`, `shift_template_id`, `shift_date`, `status`, timestamps	unique `employee_id + shift_date`, index `shift_template_id`, `status`	FK `employee_id -> employees.id`, FK `shift_template_id -> shift_templates.id`	parent of `attendance_records`, `tasks`, `shift_swap_requests`
`attendance_records`	`id`, `employee_id`, `shift_id`, `attendance_date`, `check_in_at`, `check_out_at`, `source`, `status`, timestamps	unique `employee_id + attendance_date`, index `shift_id`, `status`, date range indexes	FK `employee_id -> employees.id`, nullable FK `shift_id -> shifts.id`	child of `employees`, optionally linked to `shifts`
`tasks`	`id`, `employee_id`, `shift_id`, `title`, `description`, `due_at`, `status`, timestamps	index `employee_id`, `shift_id`, `status`, `due_at`	FK `employee_id -> employees.id`, nullable FK `shift_id -> shifts.id`	child of `employees`, optionally attached to `shifts`
`shift_swap_requests`	`id`, `requester_employee_id`, `requester_shift_id`, `target_employee_id`, `target_shift_id`, `reason`, `status`, timestamps	index `requester_employee_id`, `target_employee_id`, `status`	FKs to `employees` and `shifts`; requester and target cannot be same business row pair	links employee-to-employee swap workflow
`leave_types`	`id`, `name`, `code`, `default_days`, `status`, timestamps	unique `code`, index `status`	unique business code	parent of `leave_balances` and `leave_requests`
`leave_balances`	`id`, `employee_id`, `leave_type_id`, `balance_days`, timestamps	unique `employee_id + leave_type_id`	FK `employee_id -> employees.id`, FK `leave_type_id -> leave_types.id`	one balance row per employee and leave type
`leave_requests`	`id`, `employee_id`, `leave_type_id`, `start_date`, `end_date`, `reason`, `status`, `approved_by`, timestamps	index `employee_id`, `leave_type_id`, `status`, `start_date`, `end_date`	FKs to `employees` and `leave_types`; `end_date >= start_date`	submitted by employee, typed by leave type
`overtime_requests`	`id`, `employee_id`, `request_date`, `hours`, `reason`, `status`, `approved_by`, timestamps	index `employee_id`, `request_date`, `status`	FK `employee_id -> employees.id`; `hours > 0`	overtime request workflow for employees

Recruitment and Employee Lifecycle

Table	Core Columns	Indexes	Constraints	Relations
`job_postings`	`id`, `branch_id`, `department_id`, `title`, `vacancies`, `closing_date`, `status`, timestamps	index `branch_id`, `department_id`, `closing_date`, `status`	FKs to `branches` and `departments`; `vacancies > 0`	parent of `job_applications`
`applicants`	`id`, `full_name`, `email`	unique `email`, index `full_name`	unique applicant email unless	parent of `job_applications`

`applicants`	`id`, `full_name`, `email`, `phone`, timestamps	unique `email`, index `full_name`	unique applicant email unless business allows duplicates	parent of `job_applications`
`job_applications`	`id`, `applicant_id`, `job_posting_id`, `stage`, `status`, timestamps	unique `applicant_id` + `job_posting_id`, index `stage`, `status`	FK `applicant_id -> applicants.id`, FK `job_posting_id -> job_postings.id`	child of applicant and posting
`application_stage_history`	`id`, `job_application_id`, `from_stage`, `to_stage`, `comment`, `acted_by`, timestamps	index `job_application_id`, `to_stage`, `acted_by`	FK `job_application_id -> job_applications.id`, FK `acted_by -> employees.id`	audit trail for recruitment stage movement
`onboarding_checklists`	`id`, `employee_id`, `title`, `status`, timestamps	index `employee_id`, `status`	FK `employee_id -> employees.id`	parent of `onboarding_tasks`
`onboarding_tasks`	`id`, `onboarding_checklist_id`, `title`, `status`, `completed_by`, `completed_at`, timestamps	index `onboarding_checklist_id`, `status`	FK `onboarding_checklist_id -> onboarding_checklists.id`, nullable FK `completed_by -> employees.id`	task items for onboarding checklist
`offboarding_checklists`	`id`, `employee_id`, `title`, `status`, timestamps	index `employee_id`, `status`	FK `employee_id -> employees.id`	parent of `offboarding_tasks`
`offboarding_tasks`	`id`, `offboarding_checklist_id`, `title`, `status`, `completed_by`, `completed_at`, timestamps	index `offboarding_checklist_id`, `status`	FK `offboarding_checklist_id -> offboarding_checklists.id`, nullable FK `completed_by -> employees.id`	task items for offboarding checklist

Planning, Talent, Learning, Payroll, and Reporting

Table	Core Columns	Indexes	Constraints	Relations
`headcount_plans`	`id`, `branch_id`, `department_id`, `job_title_id`, `period`, `planned_headcount`, timestamps	unique `branch_id` + `department_id` + `job_title_id` + `period`	FKs to `branches`, `departments`, `job_titles`; `planned_headcount >= 0`	planning record by org slice
`position_forecasts`	`id`, `branch_id`, `department_id`, `job_title_id`, `period`, `forecast_headcount`, timestamps	unique `branch_id` + `department_id` + `job_title_id` + `period`	FKs to `branches`, `departments`, `job_titles`; `forecast_headcount >= 0`	forecast record by org slice
`competencies`	`id`, `name`, `description`, timestamps	unique `name`	unique competency name	parent of `competency_assessments`
`competency_assessments`	`id`, `employee_id`, `competency_id`, `rating`, `comment`, timestamps	index `employee_id`, `competency_id`, created date	FK `employee_id -> employees.id`, FK `competency_id -> competencies.id`; rating bounded by agreed scale	latest assessment powers competency profile
`kpi_definitions`	`id`, `name`, `weight`, `formula_type`, `status`, timestamps	index `status`, `formula_type`	`weight >= 0`; status enum	parent of `kpi_scores`
`scorecards`	`id`, `employee_id`, `period_start`, `period_end`, `status`, aggregate fields, timestamps	index `employee_id`, `period_start`, `period_end`, `status`	FK `employee_id -> employees.id`; one scorecard per employee and review period by business rule	parent of `kpi_scores`
`kpi_scores`	`id`, `scorecard_id`, `kpi_definition_id`, `score`, `comment`, timestamps	unique `scorecard_id` + `kpi_definition_id`	FK `scorecard_id -> scorecards.id`, FK `kpi_definition_id -> kpi_definitions.id`	score line items inside scorecard
`training_courses`	`id`, `title`, `description`, `status`, timestamps	index `status`, title search index if needed	status enum	parent of `training_enrollments`
`training_enrollments`	`id`, `course_id`, `employee_id`, `progress_percent`, `status`, timestamps	unique `course_id` + `employee_id`, index `status`	FK `course_id -> training_courses.id`, FK `employee_id -> employees.id`; `progress_percent` between 0 and 100	enrollment bridge between courses and employees
`payroll_runs`	`id`, `branch_id`, `period_start`, `period_end`, `status`, timestamps	index `branch_id`, `period_start`, `period_end`, `status`	FK `branch_id -> branches.id`; one run per branch-period by business rule	parent of `payroll_run_items`
`payroll_run_items`	`id`, `payroll_run_id`, `employee_id`, totals, timestamps	unique `payroll_run_id` + `employee_id`, index `employee_id`	FK `payroll_run_id -> payroll_runs.id`, FK `employee_id -> employees.id`	line items for payroll run
`notifications`	`id`, `employee_id`, `type`, `title`, `body`, `is_read`, `created_at`	index `employee_id`, `is_read`, `created_at`	FK `employee_id -> employees.id`	inbox rows for employees

`report_definitions`	`id`, `name`, `module`, `filters_json`, timestamps	index `module`	module constrained to known report modules	parent of `report_runs`
`report_runs`	`id`, `report_definition_id`, `requested_by`, `status`, `filters_json`, `result_json`, timestamps	index `report_definition_id`, `requested_by`, `status`, `created_at`	FK `report_definition_id -> report_definitions.id`, FK `requested_by -> employees.id`	execution history for saved report definitions

ERD Source Note

- This embedded ERD is the backend implementation view derived from the SRS-based ERD in HRIS_SRS_Based_ERD.md.
- Use this backend file for delivery decisions and the SRS ERD file when you need the original broader modeling rationale.

Endpoint Module Ownership

This split keeps backend ownership by domain. Use the master contract for full request and response examples where needed.

Module	Endpoint Surface
System and Authentication	`/api/v1/health`, `/api/v1/auth/login`, `/api/v1/auth/logout`, `/api/v1/auth/me`, `/api/v1/auth/refresh`
RBAC and Scope	`/api/v1/roles`, `/api/v1/permissions`, `/api/v1/employees/{employee}/roles`, `/api/v1/employees/{employee}/branches/sync`, `/api/v1/employees/{employee}/departments/sync`
HR Core	`/api/v1/branches`, `/api/v1/departments`, `/api/v1/job-titles`, `/api/v1/employees`, `/api/v1/contracts`
Attendance	`/api/v1/attendance/check-in`, `/api/v1/attendance/check-out`, `/api/v1/attendance`, `/api/v1/attendance/{attendance}/corrections`
Shift Management	`/api/v1/shift-templates`, `/api/v1/shift-assignments`, `/api/v1/employees/{employee}/shifts`
Leave	`/api/v1/leave-types`, `/api/v1/leave-balances`, `/api/v1/leave-requests`, `/api/v1/employees/{employee}/leave-balances/adjust`
Overtime	`/api/v1/overtime-requests`, `/api/v1/overtime-summaries/monthly`
Notifications	`/api/v1/device-tokens`, `/api/v1/notifications`, `/api/v1/notifications/read-all`
Swap Requests and Tasks	`/api/v1/swap-requests`, `/api/v1/tasks`
Recruitment	`/api/v1/job-postings`, `/api/v1/applicants`, `/api/v1/applications`, `/api/v1/applications/{application}/move-stage`, `/api/v1/applications/{application}/mark-hired`
Onboarding and Offboarding	`/api/v1/onboarding-checklists`, `/api/v1/onboarding-tasks/{task}/complete`, `/api/v1/offboarding-checklists`, `/api/v1/offboarding-tasks/{task}/complete`
Workforce Planning	`/api/v1/headcount-plans`, `/api/v1/workforce-forecasts`
Competency	`/api/v1/competencies`, `/api/v1/competency-assessments`, `/api/v1/employees/{employee}/competency-profile`
Performance and KPI	`/api/v1/kpis`, `/api/v1/scorecards`, `/api/v1/employees/{employee}/performance-summary`
Training	`/api/v1/courses`, `/api/v1/enrollments`, `/api/v1/quizzes/{quiz}/submit`
Payroll	`/api/v1/payroll-runs`, `/api/v1/payslips`
Reports	`/api/v1/report-definitions`, `/api/v1/report-runs/{reportRun}`

Laravel Request and Resource Class Matrix

Naming Rule

- Use `Index<Domain>Request` for filter-heavy list endpoints.
- Use `Store<Domain>Request` and `Update<Domain>Request` for create and update endpoints.
- Use `<Action><Domain>Request` for state transitions and one-off commands.
- Use `<Domain>Resource` for single records and `<Domain>Resource::collection(...)` for paginated lists.
- Use dedicated action classes for every approval, sync, publication, and workflow transition.

Module	Endpoints	Form Request Classes	API Resource Classes	Action or Service Classes
System and Auth	`GET /health`, `POST`	`LoginRequest`,	`HealthResource`,	`LoginAction`, `LogoutAction`,

	`/auth/login`, `POST /auth/logout`, `GET /auth/me`, `POST /auth/refresh`	`RefreshTokenRequest`	`AuthSessionResource`, `AuthProfileResource`	`RefreshTokenAction`, `GetCurrentEmployeeProfileAction`
RBAC and Scope	`GET/POST /roles`, `GET /permissions`, `POST /employees/{employee}/roles`, `PUT /employees/{employee}/branches/sync`, `PUT /employees/{employee}/departments/sync`	`IndexRoleRequest`, `StoreRoleRequest`, `AssignEmployeeRolesRequest`, `SyncEmployeeBranchesRequest`, `SyncEmployeeDepartmentsRequest`	`RoleResource`, `PermissionResource`, `EmployeeRoleAssignmentResource`, `EmployeeScopeResource`	`CreateRoleAction`, `AssignRolesToEmployeeAction`, `SyncEmployeeBranchScopeAction`, `SyncEmployeeDepartmentScopeAction`
Branches	`GET/POST /branches`, `PUT /branches/{branch}`	`IndexBranchRequest`, `StoreBranchRequest`, `UpdateBranchRequest`	`BranchResource`	`CreateBranchAction`, `UpdateBranchAction`
Departments	`GET/POST /departments`, `PUT /departments/{department}`	`IndexDepartmentRequest`, `StoreDepartmentRequest`, `UpdateDepartmentRequest`	`DepartmentResource`	`CreateDepartmentAction`, `UpdateDepartmentAction`
Job Titles	`GET/POST /job-titles`, `PUT /job-titles/{jobTitle}`	`IndexJobTitleRequest`, `StoreJobTitleRequest`, `UpdateJobTitleRequest`	`JobTitleResource`	`CreateJobTitleAction`, `UpdateJobTitleAction`
Employees	`GET/POST /employees`, `GET/PUT /employees/{employee}`	`IndexEmployeeRequest`, `StoreEmployeeRequest`, `UpdateEmployeeRequest`	`EmployeeSummaryResource`, `EmployeeDetailResource`	`CreateEmployeeAction`, `UpdateEmployeeAction`, `GenerateEmployeeCodeService`
Contracts	`GET /employees/{employee}/contracts`, `POST /employees/{employee}/contracts`, `GET/PUT /contracts/{contract}`	`IndexEmployeeContractRequest`, `StoreContractRequest`, `UpdateContractRequest`	`ContractResource`	`CreateContractAction`, `UpdateContractAction`
Attendance	`POST /attendance/check-in`, `POST /attendance/check-out`, `GET /attendance`, `GET /employees/{employee}/attendance`, `POST /attendance/{attendance}/corrections`	`CheckInAttendanceRequest`, `CheckOutAttendanceRequest`, `IndexAttendanceRequest`, `IndexEmployeeAttendanceRequest`, `CorrectAttendanceRequest`	`AttendanceResource`	`CheckInAttendanceAction`, `CheckOutAttendanceAction`, `CorrectAttendanceAction`
Shift Management	`GET/POST/PUT /shift-templates`, `GET/POST /shift-assignments`, `GET /employees/{employee}/shifts`	`IndexShiftTemplateRequest`, `StoreShiftTemplateRequest`, `UpdateShiftTemplateRequest`, `IndexShiftAssignmentRequest`, `StoreShiftAssignmentRequest`, `IndexEmployeeShiftRequest`	`ShiftTemplateResource`, `ShiftAssignmentResource`	`CreateShiftTemplateAction`, `UpdateShiftTemplateAction`, `AssignShiftAction`
Leave	`GET /leave-types`, `GET /leave-balances`, `GET /employees/{employee}/leave-balances`, `POST /leave-requests`, `GET /leave-requests`, `GET /leave-requests/{leaveRequest}`, `POST /leave-requests/{leaveRequest}/approve`, `POST /leave-requests/{leaveRequest}/reject`, `POST /employees/{employee}/leave-balances/adjust`	`IndexLeaveTypeRequest`, `IndexLeaveBalanceRequest`, `IndexEmployeeLeaveBalanceRequest`, `StoreLeaveRequestRequest`, `IndexLeaveRequestRequest`, `ApproveLeaveRequest`, `RejectLeaveRequest`, `AdjustLeaveBalanceRequest`	`LeaveTypeResource`, `LeaveBalanceResource`, `LeaveRequestResource`	`SubmitLeaveRequestAction`, `ApproveLeaveRequestAction`, `RejectLeaveRequestAction`, `AdjustLeaveBalanceAction`
Overtime	`POST /overtime-requests`, `GET /overtime-requests`, `GET /overtime-requests/{overtimeRequest}`, `POST /overtime-requests/{overtimeRequest}/approve`, `POST /overtime-requests/{overtimeRequest}/reject`, `GET /overtime-summaries/monthly`	`StoreOvertimeRequest`, `IndexOvertimeRequest`, `ApproveOvertimeRequest`, `RejectOvertimeRequest`, `MonthlyOvertimeSummaryRequest`	`OvertimeRequestResource`, `OvertimeSummaryResource`	`SubmitOvertimeRequestAction`, `ApproveOvertimeRequestAction`, `RejectOvertimeRequestAction`, `BuildMonthlyOvertimeSummaryAction`
Notifications	`POST /device-tokens`, `GET /notifications`, `POST /notifications/{notification}/read`, `POST /notifications/read-all`	`StoreDeviceTokenRequest`, `IndexNotificationRequest`, `MarkNotificationReadRequest`, `MarkAllNotificationsReadRequest`	`DeviceTokenResource`, `NotificationResource`	`RegisterDeviceTokenAction`, `MarkNotificationReadAction`, `MarkAllNotificationsReadAction`
Swap Requests and Tasks	`POST /swap-requests`, `GET /swap-requests`, `GET /swap-requests/{swapRequest}`, `POST /swap-requests/{swapRequest}/approve`, `POST /swap-`	`StoreSwapRequestRequest`, `IndexSwapRequestRequest`, `ApproveSwapRequest`, `RejectSwapRequest`, `IndexTaskRequest`, `StoreTaskRequest`,	`SwapRequestResource`, `TaskResource`	`SubmitSwapRequestAction`, `ApproveSwapRequestAction`, `RejectSwapRequestAction`, `CreateTaskAction`, `UpdateTaskAction`

	requests/{swapRequest}/reject , GET/POST /tasks , PUT /tasks/{task}	UpdateTaskRequest		
Recruitment	GET/POST/PUT /job-postings , GET/POST /applicants , GET/POST /applications , POST /applications/{application}/move-stage , POST /applications/{application}/mark-hired	IndexJobPostingRequest , StoreJobPostingRequest , UpdateJobPostingRequest , IndexApplicantRequest , StoreApplicantRequest , IndexApplicationRequest , StoreApplicationRequest , MoveApplicationStageRequest , MarkApplicationHiredRequest	JobPostingResource , ApplicantResource , ApplicationResource , HiredApplicationResource	CreateJobPostingAction , UpdateJobPostingAction , CreateApplicantAction , CreateApplicationAction , MoveApplicationStageAction , MarkApplicationHiredAction
Onboarding and Offboarding	GET/POST /onboarding-checklists , GET /onboarding-checklists/{checklist} , POST /onboarding-checklists/{checklist}/tasks , POST /onboarding-tasks/{task}/complete , GET/POST /offboarding-checklists , GET /offboarding-checklists/{checklist} , POST /offboarding-checklists/{checklist}/tasks , POST /offboarding-tasks/{task}/complete	IndexOnboardingChecklistRequest , StoreOnboardingChecklistRequest , StoreOnboardingTaskRequest , CompleteOnboardingTaskRequest , IndexOffboardingChecklistRequest , StoreOffboardingChecklistRequest , StoreOffboardingTaskRequest , CompleteOffboardingTaskRequest	OnboardingChecklistResource , OnboardingTaskResource , OffboardingChecklistResource , OffboardingTaskResource	CreateOnboardingChecklistAction , AddOnboardingTaskAction , CompleteOnboardingTaskAction , CreateOffboardingChecklistAction , AddOffboardingTaskAction , CompleteOffboardingTaskAction
Workforce Planning	GET/POST /headcount-plans , GET/POST /workforce-forecasts	IndexHeadcountPlanRequest , StoreHeadcountPlanRequest , IndexWorkforceForecastRequest , StoreWorkforceForecastRequest	HeadcountPlanResource , WorkforceForecastResource	CreateHeadcountPlanAction , CreateWorkforceForecastAction
Competency	GET/POST /competencies , GET/POST /competency-assessments , GET /employees/{employee}/competency-profile	IndexCompetencyRequest , StoreCompetencyRequest , IndexCompetencyAssessmentRequest , StoreCompetencyAssessmentRequest , EmployeeCompetencyProfileRequest	CompetencyResource , CompetencyAssessmentResource , CompetencyProfileResource	CreateCompetencyAction , CreateCompetencyAssessmentAction , BuildEmployeeCompetencyProfileAction
Performance and KPI	GET/POST /kpis , GET /scorecards , GET /scorecards/{scorecard} , POST /scorecards/{scorecard}/scores , GET /employees/{employee}/performance-summary	IndexKpiRequest , StoreKpiRequest , IndexScorecardRequest , SubmitScorecardScoresRequest , EmployeePerformanceSummaryRequest	KpiResource , ScorecardResource , PerformanceSummaryResource	CreateKpiAction , SubmitScorecardScoresAction , BuildPerformanceSummaryAction
Training	GET/POST /courses , GET /courses/{course} , POST /courses/{course}/enrollments , GET /enrollments , POST /enrollments/{enrollment}/progress , POST /quizzes/{quiz}/submit	IndexCourseRequest , StoreCourseRequest , StoreCourseEnrollmentRequest , IndexEnrollmentRequest , UpdateEnrollmentProgressRequest , SubmitQuizRequest	CourseResource , EnrollmentResource , QuizSubmissionResource	CreateCourseAction , EnrollEmployeeInCourseAction , UpdateEnrollmentProgressAction , SubmitQuizAction
Payroll	GET/POST /payroll-runs , GET /payroll-runs/{payrollRun} , POST /payroll-runs/{payrollRun}/approve , GET /payroll-runs/{payrollRun}/line-items , GET /payslips , GET /payslips/{payslip}	IndexPayrollRunRequest , StorePayrollRunRequest , ApprovePayrollRunRequest , IndexPayrollLineItemRequest , IndexPayslipRequest	PayrollRunResource , PayrollLineItemResource , PayslipResource	CreatePayrollRunAction , ApprovePayrollRunAction , GeneratePayrollLineItemsAction
Reports	GET/POST /report-definitions , GET /report-definitions/{reportDefinition} , POST /report-definitions/{reportDefinition}/run , GET /report-runs/{reportRun}	IndexReportDefinitionRequest , StoreReportDefinitionRequest , RunReportDefinitionRequest , ShowReportRunRequest	ReportDefinitionResource , ReportRunResource	CreateReportDefinitionAction , RunReportDefinitionAction

Backend Folder Placement

- `app/Http/Requests/Api/V1/<Module>/...`
- `app/Http/Resources/Api/V1/<Module>/...`
- `app/Actions/<Module>/...`

- `app/Policies/...`
- `app/Services/<Module>/...` only when logic is cross-action or infrastructure-oriented.

Database Migration and Seeder Matrix

Migration Rules

- Use timestamped Laravel migrations under `database/migrations` with one responsibility per migration.
- Create lookup and structure tables before transactional tables.
- Add foreign keys explicitly and prefer `cascadeOnUpdate()` with deliberate delete behavior.
- Use PostgreSQL enums only if the team is committed to DB-level enum maintenance; otherwise store enum-like values as indexed strings validated in code.
- Every table with workflow relevance should include audit-friendly timestamps and actor references where needed.

Domain	Core Tables	Key Columns	Foreign Keys and Indexes	Seeder Classes
Access Control	`roles`, `permissions`, `model_has_roles`, `model_has_permissions`, `role_has_permissions`	Spatie standard columns plus guard name	Index `name + guard_name`; polymorphic indexes per Spatie defaults	`PermissionSeeder`, `RoleSeeder`, `RolePermissionSeeder`
Organization	`branches`, `departments`, `job_titles`	`branches.code`, `branches.name`, `departments.parent_id`, `job_titles.department_id`, `status`	Unique `branches.code`; index `status`; FKs `departments.branch_id`, `departments.parent_id`, `job_titles.department_id`	`BranchSeeder`, `DepartmentSeeder`, `JobTitleSeeder`
Employees	`employees`, `employee_branch_scopes`, `employee_department_scopes`	`employee_code`, `full_name`, `email`, `phone`, `hire_date`, `status`	Unique `employee_code`, `email`; FKs to branch, department, job title; indexes on `status`, `branch_id`, `department_id`, `job_title_id`	`EmployeeSeeder`, `EmployeeScopeSeeder`
Contracts	`contracts`	`employee_id`, `type`, `start_date`, `end_date`, `salary`, `currency`, `status`	FK `employee_id`; index `status`; partial or composite index on active contracts per employee if needed	`ContractSeeder`
Attendance	`attendance_records`, `attendance_corrections`	`employee_id`, `attendance_date`, `check_in_at`, `check_out_at`, `status`, geo fields, `source`	Unique or constrained index on `employee_id + attendance_date`; FK correction to record and approver; indexes on date range and status	`AttendanceSeeder`
Shift Management	`shift_templates`, `shift_assignments`	template start and end times, break minutes, assignment date, status	Unique `employee_id + date` if only one shift per day; FKs to employee and template; indexes on branch lookup through employee if needed	`ShiftTemplateSeeder`, `ShiftAssignmentSeeder`
Leave	`leave_types`, `leave_balances`, `leave_requests`, `leave_request_actions`	leave day allocations, request dates, status, approver comments	FKs to employee, leave type, approver; indexes on `status`, `start_date`, `end_date`; unique balance per `employee_id + leave_type_id`	`LeaveTypeSeeder`, `LeaveBalanceSeeder`, `LeaveRequestSeeder`
Overtime	`overtime_requests`	`employee_id`, `date`, `hours`, `reason`, `status`, approver metadata	FKs to employee and approver; indexes on `date`, `status`, `employee_id`	`OvertimeRequestSeeder`
Notifications	`notifications`, `device_tokens`	Laravel notification payload columns, `platform`, `device_token`, `is_read`	FK `employee_id` on device tokens if modeled explicitly; index `is_read`, created date; unique `device_token` when appropriate	`NotificationSeeder`, `DeviceTokenSeeder`
Swap Requests and Tasks	`swap_requests`, `tasks`	requester and target shift references, reason, due date, task status	FKs to shifts, employees, assignee; indexes on `status`, `employee_id`, `shift_id`, `due_at`	`SwapRequestSeeder`, `TaskSeeder`
Recruitment	`job_postings`, `applicants`, `applications`, `application_stage_logs`	vacancies, closing date, applicant contacts, application stage, stage comments	FKs to branch, department, applicant, posting; indexes on `status`, `stage`, `closing_date`	`JobPostingSeeder`, `ApplicantSeeder`, `ApplicationSeeder`
Onboarding and Offboarding	`onboarding_checklists`, `onboarding_tasks`, `offboarding_checklists`, `offboarding_tasks`	checklist title, task title, task status, completion note	FKs to employee, checklist, completer; indexes on `status`, `employee_id`	`OnboardingChecklistSeeder`, `OffboardingChecklistSeeder`
Workforce Planning	`headcount_plans`, `workforce_forecasts`	`period`, `planned_headcount`, `forecast_headcount`	FKs to branch, department, job title; unique composite on `branch_id + department_id + job_title_id + period`	`HeadcountPlanSeeder`, `WorkforceForecastSeeder`

Competency	`competencies`, `competency_assessments`	`name`, `description`, `rating`, `comment`	FKs to employee and competency; indexes on `employee_id`, `competency_id`, created date	`CompetencySeeder`, `CompetencyAssessmentSeeder`
Performance	`kpis`, `scorecards`, `scorecard_scores`	`weight`, `formula_type`, review period, aggregate score	FKs to employee, KPI, scorecard; indexes on period and employee scope	`KpiSeeder`, `ScorecardSeeder`
Training	`courses`, `course_modules`, `enrollments`, `quizzes`, `quiz_questions`, `quiz_attempts`	publication status, progress percent, quiz score, pass flag	FKs to course, enrollment, employee; indexes on `status`, `employee_id`, `course_id`	`CourseSeeder`, `EnrollmentSeeder`, `QuizSeeder`
Payroll	`payroll_runs`, `payroll_line_items`, `payslips`, `payslip_items`	period start and end, run status, totals, earning and deduction types	FKs to run, employee, payslip; indexes on `status`, `branch_id`, period columns	`PayrollRunSeeder`, `PayslipSeeder`
Reports	`report_definitions`, `report_runs`	report module, filter schema JSON, run status, row payload JSONB	Index `module`, `status`, created date; FK actor references if tracked	`ReportDefinitionSeeder`
Audit	`audit_logs` or vendor package tables	actor, event, auditable type, before and after data	Index `auditable_type + auditable_id`, `actor_id`, `created_at`	`AuditEventTypeSeeder` only if custom lookup data is needed

Recommended Seeder Order

1. `PermissionSeeder`
2. `RoleSeeder`
3. `RolePermissionSeeder`
4. `BranchSeeder`
5. `DepartmentSeeder`
6. `JobTitleSeeder`
7. `LeaveTypeSeeder`
8. `ShiftTemplateSeeder`
9. `CompetencySeeder`
10. `KpiSeeder`
11. `CourseSeeder`
12. `ReportDefinitionSeeder`
13. `EmployeeSeeder`
14. `EmployeeScopeSeeder`
15. `ContractSeeder`
16. Remaining transactional demo seeders only for non-production environments

Seeder Environment Rule

- Production seeders should include only baseline reference data, permissions, roles, and optionally one bootstrap super admin account.
- Demo or QA data belongs in dedicated environment-gated seeders such as `Database\\Seeders\\Demo\\...`.
- Payroll, attendance, leave requests, and other sensitive transactional data must not be seeded into production.

Permission Seed Matrix

Spatie Seeding Rule

- Seed permissions first, then roles, then attach permissions to roles, then attach roles to employees.
- Use a single guard name for API-authenticated users, for example `sanctum` or `web`, and keep it consistent across all seeded permissions and roles.
- Treat permissions as the source of truth for backend authorization; roles are only bundles of permissions.
- Prefer explicit permission strings in `snake.case` format exactly as documented by the API contract.

Permission Catalog by Module

Module	Permissions
RBAC and Scope	`roles.view`, `roles.create`, `permissions.view`, `employees.assign_roles`, `employees.manage_scope`
Organization	`branches.view`, `branches.create`, `branches.update`, `departments.view`, `departments.create`, `departments.update`, `job_titles.view`, `job_titles.create`, `job_titles.update`
Employees and Contracts	`employees.view`, `employees.create`, `employees.update`, `contracts.view`, `contracts.create`, `contracts.update`
Attendance and Shifts	`attendance.check_in`, `attendance.check_out`, `attendance.view`, `attendance.correct`, `shifts.view`, `shifts.create`, `shifts.update`, `shifts.assign`

Leave and Overtime	`leave.view`, `leave.view_balances`, `leave.request`, `leave.approve`, `leave.adjust_balance`, `overtime.request`, `overtime.view`, `overtime.approve`
Notifications and Self Service	no additional admin permission required for self inbox and token registration in month-1 MVP
Swap Requests and Tasks	`swap.request`, `swap.view`, `swap.approve`, `tasks.view`, `tasks.create`, `tasks.update`
Recruitment	`recruitment.view`, `recruitment.create`, `recruitment.update`, `recruitment.hire`
Onboarding and Offboarding	`onboarding.view`, `onboarding.create`, `onboarding.update`, `offboarding.view`, `offboarding.create`, `offboarding.update`
Planning	`planning.view`, `planning.create`
Competency and Performance	`competencies.view`, `competencies.create`, `competencies.assess`, `performance.view`, `performance.create`, `performance.score`
Training	`training.view`, `training.create`, `training.enroll`
Payroll and Reports	`payroll.view`, `payroll.create`, `payroll.approve`, `reports.view`, `reports.create`, `reports.run`

Baseline Role Matrix

Role	Intended Users	Seeded Permissions
`super_admin`	Initial system owner	All permissions
`employee`	Every authenticated employee	`attendance.check_in`, `attendance.check_out`, `leave.request`, `overtime.request`, `swap.request`
`hr_manager`	HR leadership	`employees.view`, `employees.create`, `employees.update`, `contracts.view`, `contracts.create`, `contracts.update`, `branches.view`, `departments.view`, `job_titles.view`, `leave.view`, `leave.view_balances`, `leave.approve`, `leave.adjust_balance`, `attendance.view`, `attendance.correct`, `reports.view`, `training.view`, `competencies.view`, `performance.view`, `onboarding.view`, `offboarding.view`
`branch_manager`	Branch-level operations lead	`employees.view`, `attendance.view`, `attendance.correct`, `shifts.view`, `shifts.assign`, `leave.view`, `leave.approve`, `overtime.view`, `overtime.approve`, `swap.view`, `swap.approve`, `tasks.view`, `tasks.create`, `tasks.update`, `reports.view`
`recruiter`	Recruitment team	`recruitment.view`, `recruitment.create`, `recruitment.update`, `recruitment.hire`, `employees.view`, `departments.view`, `job_titles.view`
`payroll_officer`	Payroll processing team	`employees.view`, `contracts.view`, `attendance.view`, `leave.view`, `overtime.view`, `payroll.view`, `payroll.create`, `payroll.approve`, `reports.view`, `reports.run`
`learning_and_performance_officer`	Training and performance team	`training.view`, `training.create`, `training.enroll`, `competencies.view`, `competencies.create`, `competencies.assess`, `performance.view`, `performance.create`, `performance.score`, `reports.view`
`organization_admin`	Org setup and access administrator	`roles.view`, `roles.create`, `permissions.view`, `employees.assign_roles`, `employees.manage_scope`, `branches.view`, `branches.create`, `branches.update`, `departments.view`, `departments.create`, `departments.update`, `job_titles.view`, `job_titles.create`, `job_titles.update`

Seeder Class Plan

- `PermissionSeeder` creates all permission strings from the catalog above.
- `RoleSeeder` creates `super_admin`, `employee`, `hr_manager`, `branch_manager`, `recruiter`, `payroll_officer`, `learning_and_performance_officer`, and `organization_admin`.
- `RolePermissionSeeder` maps role bundles using `syncPermissions()`.
- `AdminUserSeeder` creates the first super admin employee and assigns `super_admin`.
- `EmployeeRoleSeeder` applies default `employee` role to normal seeded employees.

Seeder Implementation Rule

- Wrap role and permission seeding in idempotent calls such as `Permission::findOrCreate()` and `Role::findOrCreate()`.
- Clear cached Spatie permissions before and after seeding via `PermissionRegistrar`.
- Keep role-to-permission mappings in one source file or config array so tests can assert them directly.

Delivery Notes

Backend Must Stay Clean By Design

- No business rules in controllers.
- No policy logic in UI code.
- No duplicate validation rules across clients and backend as source-of-truth.
- No raw database shapes leaked to clients.
- No module-specific response envelopes.

Working Rule

- The master contract remains the canonical cross-team source.
- This backend split is the day-to-day implementation guide for backend delivery.